Light

Algorithms — Trace Tables & Dry Runs

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Algorithms and problem-solving
2. Representing algorithms
3. Trace tables
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Algorithms and Trace Tables (8525 Paper 1)**. This topic covers algorithm definitions, decomposition, abstraction, representing algorithms using pseudo-code, flowcharts and program code, and using trace tables to determine algorithm purpose. You must be able to define, explain, trace and analyse algorithms in exam questions.

Exam questions typically ask you to **define** what an algorithm is, **explain** decomposition and abstraction, **complete** trace tables, **determine** the purpose of algorithms, and **identify** inputs, processing and outputs. Questions range from 1-mark definitions to multi-mark trace table completions and algorithm analysis tasks.

Why algorithms matter

An **algorithm** is a sequence of steps that can be followed to complete a task. Algorithms are the foundation of all computer programs — before any code is written, the problem must be understood and a step-by-step solution designed. This planning process uses techniques like decomposition (breaking problems down) and abstraction (removing unnecessary detail).

Understanding algorithms helps you think logically, solve problems systematically, and write better programs. In the AQA exam, you will need to read algorithms

written in pseudo-code or program code, trace through them step by step using trace tables, and determine what they do and why.

- **Algorithm:** A sequence of steps to complete a task
- **Decomposition:** Breaking a problem into smaller sub-problems
- **Abstraction:** Removing unnecessary detail from a problem
- **Trace table:** A technique for tracking variable values as an algorithm executes

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point and demonstrate each skill.

- **Understand and explain** the term algorithm.
- **Be aware** that a computer program is an implementation of an algorithm and that an algorithm is not a computer program.
- **Understand and explain** the term decomposition.
- **Understand and explain** the term abstraction.
- **Use a systematic approach** to problem solving and algorithm creation representing those algorithms using pseudo-code, program code and flowcharts.
- **Explain** simple algorithms in terms of their inputs, processing and outputs.
- **Determine** the purpose of simple algorithms using trace tables and visual inspection.

> **Specification note.** AQA 8525 assesses algorithms on Paper 1 (Computational thinking and programming skills). Any exam question where students are given pseudo-code will use the AQA standard version. Exam questions will indicate the form of response expected — for example, pseudo-code, program code or a flowchart. Students must respond as instructed.

Toolkit

Key definitions at a glance

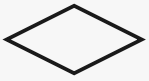
Term	Definition
Algorithm	A sequence of steps that can be followed to complete a task.
Decomposition	Breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be further subdivided.
Abstraction	The process of removing unnecessary detail from a problem.
Trace table	A technique for recording the values of variables as an algorithm is executed step by step.
Inputs	Data that goes into an algorithm (from user, file, sensor, etc.).
Processing	The operations performed on data (calculations, comparisons, decisions).

Term	Definition
Outputs	Data that comes out of an algorithm (displayed, stored, sent, etc.).

AQA pseudo-code conventions

Construct	AQA pseudo-code	Python equivalent
Assignment	x ← 5	x = 5
Input	x ← USERINPUT	x = input()
Output	OUTPUT x	print(x)
Selection (IF)	IF condition THEN ... ENDIF	if condition:
Selection (IF-ELSE)	IF ... THEN ... ELSE ... ENDIF	if ... else:
Count-controlled loop	FOR i ← 1 TO 5 ... ENDFOR	for i in range(1, 6):
Condition-controlled loop	WHILE condition ... ENDWHILE	while condition:
Integer division	x DIV y	x // y
Remainder (modulo)	x MOD y	x % y
Array access	arr[0]	arr[0]

Flowchart symbols

Symbol	Name	Purpose
	Terminator	Start or End of algorithm
	Process	Calculation, assignment, action
	**Decision**	Yes/No question, condition check
	Input/Output	Data entering or leaving the algorithm

Symbol	Name	Purpose
⟶	**Flow line**	Direction of flow between steps

Trace table template

A trace table has one column for each variable (and optionally for outputs). Each row represents one step or iteration of the algorithm.

Step	Variable 1	Variable 2	Variable 3	OUTPUT
1	initial value	—	—	—
2	updated value	initial value	—	—
3	...	...	...	...

1. Algorithms and problem-solving

Before writing any program, a programmer must first understand the problem and design a solution. This solution is called an algorithm. Understanding algorithms is fundamental to computational thinking and is heavily tested in the AQA exam.

1.1 What is an algorithm?

An **algorithm** is a sequence of steps that can be followed to complete a task. Algorithms exist in everyday life — a recipe for making a cake is an algorithm, as are the instructions for assembling furniture or directions to a location.

Definition: An algorithm is a sequence of steps that can be followed to complete a task.

Key characteristics of algorithms:

- **Finite:** The algorithm must eventually stop (it cannot run forever)
- **Precise:** Each step must be clearly defined and unambiguous
- **Ordered:** Steps must be performed in a specific sequence
- **Effective:** Each step must be achievable

1.2 Algorithms vs programs

It is important to understand the distinction between an algorithm and a computer program:

- An **algorithm** is a logical design — a plan for solving a problem
- A **computer program** is an implementation of an algorithm — the algorithm written in a specific programming language that a computer can execute

An algorithm is not a computer program. You can have an algorithm without ever writing code — it might be written in pseudo-code, drawn as a flowchart, or even described in plain English. However, a computer program is always based on an underlying algorithm.

> **Exam focus.** AQA may ask you to distinguish between algorithms and programs. Remember: an algorithm is the plan/design; a program is the code that implements it.

1.3 Decomposition

Decomposition means breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be further subdivided.

> **Definition:** Decomposition is breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be further subdivided.

Decomposition is essential for tackling complex problems. By breaking a large problem into smaller, manageable parts, each part can be:

- Solved independently
- Tested separately
- Assigned to different team members
- Reused in other projects

Example: Creating a computer game could be decomposed into sub-problems such as: designing the user interface, handling player input, managing game physics, creating enemy AI, playing sound effects, and saving high scores. Each of these could be further decomposed — for example, "handling player input" might include: detecting keyboard presses, detecting mouse clicks, and interpreting controller buttons.

1.4 Abstraction

Abstraction is the process of removing unnecessary detail from a problem. It allows us to focus on the important aspects while ignoring irrelevant complexity.

> **Definition:** Abstraction is the process of removing unnecessary detail from a problem.

Abstraction helps us create simpler models of complex systems. Examples include:

- A map is an abstraction of a geographical area — it shows roads and landmarks but not every tree, building or person
- A flight simulator abstracts the experience of flying — it models the important controls and physics but not the molecular composition of the aircraft
- A variable in programming is an abstraction — it represents a memory location without the programmer needing to know the exact address

When designing algorithms, abstraction helps us focus on what the algorithm needs to do without getting distracted by implementation details that can be worked out later.

1.5 Inputs, processing and outputs

Every algorithm can be analysed in terms of three components:

- **Inputs:** Data that enters the algorithm. This might come from the user (keyboard, mouse), from files, from sensors, from other programs, or from within the program itself (constants, stored values)
- **Processing:** The operations performed on the data. This includes calculations, comparisons, decisions, and any manipulation of data

- **Outputs:** Data that leaves the algorithm. This might be displayed on screen, saved to a file, sent to another device, or used to control hardware
  Understanding inputs, processing and outputs helps you analyse what an algorithm does and how data flows through it.

Worked Example 1 — Defining an algorithm

Question: Define what is meant by an algorithm. [1 mark]

Model answer:
A sequence of steps that can be followed to complete a task **(1)**.

Exam tip: Learn this definition exactly as it appears in the AQA specification. "Sequence of steps" and "complete a task" are the key phrases.

Worked Example 2 — Algorithms vs programs

Question: Explain the difference between an algorithm and a computer program. [2 marks]

Model answer:
An algorithm is a sequence of steps/plan/design for solving a problem **(1)**. A computer program is an implementation of an algorithm / the algorithm written in a programming language that can be executed by a computer **(1)**.

Exam tip: Key distinction: algorithm = design/plan; program = implementation in code.

Worked Example 3 — Explaining decomposition

Question: A student is creating a program to manage a library system. Explain what is meant by decomposition and give an example of how it could be applied to this problem. [3 marks]

Model answer:
Decomposition is breaking a problem into smaller sub-problems **(1)**, where each sub-problem accomplishes an identifiable task **(1)**. For the library system, sub-problems could include: managing book records, handling member registrations, processing loans and returns, calculating fines, and generating reports **(1)**.

Exam tip: When asked to give an example, make sure it relates to the context given in the question (here: library system). Generic examples may not score

full marks.

Worked Example 4 — Explaining abstraction

Question: Explain what is meant by abstraction in computational thinking. [2 marks]

Model answer:
Abstraction is the process of removing unnecessary detail from a problem **(1)**. It allows focus on the important/relevant aspects while ignoring complexity that is not needed **(1)**.

Exam tip: "Removing unnecessary detail" is the key phrase. You can extend your answer by explaining why this is useful (focus, simplification).

2. Representing algorithms

Algorithms can be represented in several ways. AQA expects you to understand and use three main methods: pseudo-code, flowcharts, and program code. The exam will specify which form you should use in your response.

2.1 Pseudo-code

Pseudo-code is a way of writing algorithms that looks similar to programming code but is not specific to any programming language. It uses structured English combined with programming-style constructs.

AQA has a standard pseudo-code format that will be used in exam questions. Key features include:

- Keywords are written in UPPERCASE (IF, THEN, WHILE, FOR, OUTPUT, etc.)
- Assignment uses the left arrow symbol: x ← 5
- User input is represented as USERINPUT
- Blocks are ended with matching keywords (ENDIF, ENDWHILE, ENDFOR)
- Comments explain what code does

Example: AQA pseudo-code

```
total ← 0
FOR i ← 1 TO 5
    num ← USERINPUT
    total ← total + num
ENDFOR
average ← total / 5
OUTPUT average
```

This algorithm inputs five numbers, calculates their total, and outputs the average.

2.2 Flowcharts

A **flowchart** is a visual representation of an algorithm using standard symbols connected by arrows. Flowcharts are useful for showing the flow of control, especially when there are decisions and loops.

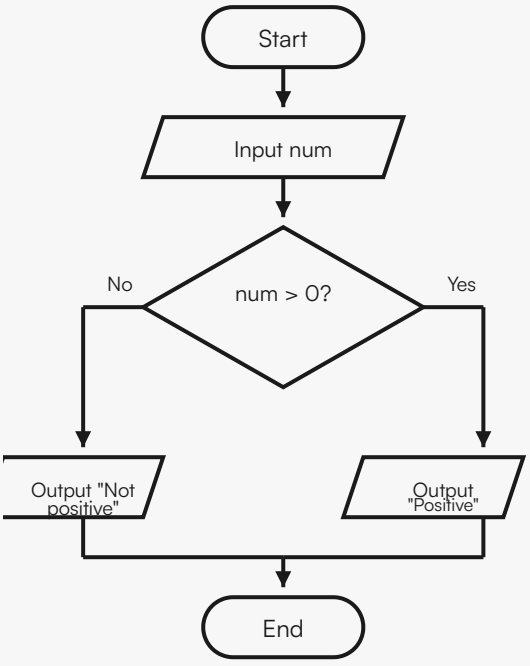
The standard flowchart symbols are:

- **Terminator (oval):** Marks the Start or End of the algorithm
- **Process (rectangle):** Represents an action, calculation or assignment
- **Decision (diamond):** Represents a Yes/No question or condition check. Has two exits: Yes and No
- **Input/Output (parallelogram):** Represents data entering or leaving the algorithm
- **Flow line (arrow):** Shows the direction of flow from one step to the next

> **Drawing flowcharts in exams.** If asked to draw a flowchart, use the correct symbols, label all arrows from decisions (Yes/No), and ensure the flow is logical and complete. Every flowchart needs a Start and an End.

Example flowchart: Check if a number is positive

This flowchart shows an algorithm that inputs a number and outputs whether it is positive or not.



2.3 Program code

AQA supports three programming languages for Paper 1: C#, Python 3, and VB.NET. On Williams Physics Education, all code examples use **Python 3**.

The algorithm shown in pseudo-code above would look like this in Python:

Example: Python code

```
total = 0
for i in range(1, 6):
    num = int(input("Enter a number: "))
```

```
    total = total + num
average = total / 5
print(average)
```

Notice the differences from pseudo-code:

- Assignment uses = instead of ←
- range(1, 6) produces 1, 2, 3, 4, 5 (the end value is exclusive)
- input() is used for user input; int() converts the text to a number
- print() is used for output
- Indentation shows which lines are inside the loop

2.4 Translating between representations

A key skill is being able to convert algorithms between pseudo-code, flowcharts and program code. The underlying logic stays the same — only the representation changes.

Selection in AQA pseudo-code

```
IF score >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Fail"
ENDIF
```

Selection in Python

```
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

While loop in AQA pseudo-code

```
count ← 0
WHILE count < 10
    OUTPUT count
    count ← count + 1
ENDWHILE
```

While loop in Python

```
count = 0
while count < 10:
    print(count)
    count = count + 1
```

Worked Example 5 — Identifying inputs, processing and outputs

Question: The following algorithm calculates the area of a rectangle:

```
length ← USERINPUT
width ← USERINPUT
area ← length * width
OUTPUT area
```

Identify the inputs, processing and outputs of this algorithm. [3 marks]

Model answer:
Inputs: length and width (entered by the user) **(1)**
Processing: Calculating the area by multiplying length by width **(1)**
Outputs: The calculated area **(1)**

Exam tip: Look for USERINPUT or input() for inputs, calculations and decisions for processing, and OUTPUT or print() for outputs.

Worked Example 6 — Converting pseudo-code to Python

Question: Convert the following AQA pseudo-code into Python. [3 marks]

```
num ← USERINPUT
IF num MOD 2 = 0 THEN
    OUTPUT "Even"
ELSE
    OUTPUT "Odd"
ENDIF
```

Model answer:

```
num = int(input())
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Exam tip: Remember: MOD becomes %, USERINPUT becomes input() (with int() if a number is needed), = in conditions becomes ==, and indentation replaces ENDIF.

Worked Example 7 — Writing an algorithm in pseudo-code

Question: Write an algorithm in pseudo-code that inputs 10 numbers and outputs the largest number entered. [4 marks]

Model answer:

```
largest ← USERINPUT
FOR i ← 2 TO 10
    num ← USERINPUT
    IF num > largest THEN
        largest ← num
    ENDIF
ENDFOR
OUTPUT largest
```

Exam tip: There are multiple correct approaches. The key is: initialise largest with the first input, then loop through remaining inputs and update largest if a bigger number is found. Always OUTPUT the result at the end.

Worked Example 8 — Describing an algorithm's purpose

Question: Describe what the following algorithm does. [2 marks]

```
word ← USERINPUT
length ← LEN(word)
FOR i ← length - 1 TO 0 STEP -1
    OUTPUT word[i]
```

```
ENDFOR
```

Model answer:
The algorithm inputs a word **(1)** and outputs the characters in reverse order / outputs the word backwards **(1)**.

Exam tip: When describing purpose, focus on WHAT the algorithm achieves overall, not a line-by-line description. Look at the inputs and outputs to understand the transformation.

3. Trace tables

A **trace table** is a technique used to track the values of variables as an algorithm executes. By working through an algorithm step by step and recording how variables change, you can determine exactly what the algorithm does and predict its output for given inputs.

3.1 What is a trace table?

A trace table has one column for each variable in the algorithm, plus an optional column for outputs. Each row represents one step or iteration where a variable changes. You work through the algorithm line by line, recording any changes to variable values.

Trace tables help you:

- Determine the purpose of an algorithm by seeing what it produces
- Find errors (bugs) in algorithms by checking if values are as expected
- Understand how loops and conditions work
- Predict outputs for given inputs

3.2 How to complete a trace table

Follow these steps to complete a trace table:

1. **Draw the table:** Create a column for each variable mentioned in the algorithm, plus an OUTPUT column if needed
2. **Start at the beginning:** Work through the algorithm from the first line
3. **Record initial values:** When a variable is first assigned a value, write it in that column
4. **Work line by line:** For each line that changes a variable, add a new row and record the new value
5. **Follow the logic:** For IF statements, check the condition and follow the correct branch. For loops, repeat the loop body as needed
6. **Record outputs:** When an OUTPUT statement is reached, write the output value in the OUTPUT column

3.3 Trace table example – simple loop

Consider this algorithm:

```
total ← 0
FOR i ← 1 TO 3
```

```
    total ← total + i
ENDFOR
OUTPUT total
```

Here is the trace table:

i	total	OUTPUT
—	0	—
1	1	—
2	3	—
3	6	—
—	—	6

The algorithm calculates 1 + 2 + 3 = 6. The purpose is to calculate the sum of numbers from 1 to 3.

3.4 Trace table example – with selection

Consider this algorithm with input values 4, 7, 2:

```
max ← 0
FOR i ← 1 TO 3
    num ← USERINPUT
    IF num > max THEN
        max ← num
    ENDIF
ENDFOR
OUTPUT max
```

Here is the trace table with inputs 4, 7, 2:

i	num	max	OUTPUT
—	—	0	—
1	4	4	—
2	7	7	—
3	2	7	—
—	—	—	7

Note: When i = 3 and num = 2, the condition num > max (2 > 7) is false, so max does not change. The algorithm finds the maximum of the three input values.

3.5 Trace table example – nested structures

Consider this algorithm that uses nested selection:

```
num ← USERINPUT
IF NOT(num > 1) OR num > 20 THEN
    OUTPUT "False"
ELSEIF num > 1 AND num < 15 THEN
    OUTPUT "Almost"
ELSEIF num MOD 5 = 0 THEN
    OUTPUT "True"
ELSE
    OUTPUT "Unknown"
ENDIF
```

This is from an actual AQA exam. Let's trace it for input num = 10:

- NOT(10 > 1) = NOT(True) = False. 10 > 20 = False. False OR False = False. First condition is false.
- 10 > 1 = True. 10 < 15 = True. True AND True = True. Second condition is true.
- Output: "Almost"

For input num = 20:

- NOT(20 > 1) = False. 20 > 20 = False. False OR False = False. First condition is false.
- 20 > 1 = True. 20 < 15 = False. True AND False = False. Second condition is false.
- 20 MOD 5 = 0? Yes, 20 MOD 5 = 0. Third condition is true.
- Output: "True"

3.6 Arrays in trace tables

When tracing algorithms that use arrays, you can either create a column for each array element (arr[0], arr[1], arr[2], etc.) or show the entire array state in one column.

Consider this algorithm:

```
days ← [10, 15, 4]
sales ← [20, 33, 12]
weeks ← [0, 0, 0]
FOR i ← 0 TO 2
    daysTotal ← days[i] + sales[i]
    weeks[i] ← daysTotal DIV 7
ENDFOR
weeksTotal ← weeks[0] + weeks[1] + weeks[2]
OUTPUT weeksTotal
```

Trace table (showing individual array accesses):

i	days[i]	sales[i]	daysTotal	weeks[i]	weeksTotal	OUTPUT
0	10	20	30	4	—	—
1	15	33	48	6	—	—
2	4	12	16	2	—	—
—	—	—	—	—	12	12

Working: weeks = [4, 6, 2], so weeksTotal = 4 + 6 + 2 = 12.

Worked Example 9 — Completing a trace table

Question: Complete the trace table for the following algorithm. [4 marks]

```
x ← 2
y ← 5
FOR i ← 1 TO 3
    x ← x + y
    y ← y - 1
ENDFOR
OUTPUT x
```

Model answer:

i	x	y	OUTPUT
—	2	5	—
1	7	4	—
2	11	3	—
3	14	2	—
—	—	—	14

Working: i=1: x = 2+5 = 7, y = 5-1 = 4. i=2: x = 7+4 = 11, y = 4-1 = 3. i=3: x = 11+3 = 14, y = 3-1 = 2. Output: 14.

Exam tip: Work through each iteration carefully. The order of operations within the loop matters — x uses the current value of y, then y changes.

Worked Example 10 – Determining algorithm purpose from trace table

Question: By completing a trace table for the input value 24, determine the output and state the purpose of this algorithm. [4 marks]

```
num ← USERINPUT
count ← 0
WHILE num > 0
    count ← count + 1
    num ← num DIV 10
ENDWHILE
OUTPUT count
```

Trace table:

num	count	OUTPUT
24	0	—
2	1	—
0	2	—
—	—	2

Output: 2 **(1)**
Purpose: The algorithm counts the number of digits in the input number **(1)**.

Exam tip: DIV 10 removes the last digit of a number. Counting how many times you can do this tells you how many digits there are.

Worked Example 11 – Trace table with Boolean conditions

Question: Complete the trace table for the following algorithm when the input is 17. [3 marks]

```
num ← USERINPUT
IF num >= 10 AND num <= 20 THEN
    OUTPUT "In range"
ELSE
    OUTPUT "Out of range"
ENDIF
```

Working:
num = 17
Check: num >= 10 → 17 >= 10 → True
Check: num <= 20 → 17 <= 20 → True
True AND True = True
Condition is true, so OUTPUT "In range"

num	OUTPUT
17	In range

Exam tip: For compound conditions (AND, OR), evaluate each part separately first, then combine. Show your working for complex conditions.

Worked Example 12 — Trace table with subroutine calls

Question: Trace through the following algorithm and state the final output when First(3, 4, 5) is called. [5 marks]

```
def First(p1, p2, p3):
    v1 = p2 + p3
    print(Second(v1, p1))

def Second(p1, p2):
    v1 = p1 + p2
    if v1 > 12:
        v1 = v1 + Third(p1)
    return v1

def Third(p1):
    if p1 > 3:
        return 2
    else:
        return 0
```

Working:
First(3, 4, 5): p1=3, p2=4, p3=5
v1 = 4 + 5 = 9
Call Second(9, 3)
Second(9, 3): p1=9, p2=3
v1 = 9 + 3 = 12

Check: v1 > 12 → 12 > 12 → False
Condition is false, so Third is not called
return 12
Output: 12

Exam tip: When tracing subroutines, keep track of which function you are in and what the parameter values are. The condition 12 > 12 is false (not >=), so the Third function is not called.

Worked examples summary

This page contains 12 worked examples covering all the key skills for algorithms and trace tables. Use this table to find specific types of questions for revision.

Example	Topic	Question type	Marks
1	Defining an algorithm	Define	1
2	Algorithms vs programs	Explain difference	2
3	Decomposition	Explain with example	3
4	Abstraction	Explain	2
5	Inputs, processing, outputs	Identify	3
6	Converting pseudo-code to Python	Convert	3
7	Writing an algorithm	Write pseudo-code	4
8	Describing algorithm purpose	Describe	2
9	Completing a trace table	Complete trace table	4
10	Determining purpose from trace	Trace and state purpose	4
11	Boolean conditions	Trace with AND/OR	3
12	Subroutine tracing	Trace function calls	5

Quick reference

Key definitions

Term	Definition (mark-scheme style)
Algorithm	A sequence of steps that can be followed to complete a task.
Decomposition	Breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be

Term	Definition (mark-scheme style)
	further subdivided.
Abstraction	The process of removing unnecessary detail from a problem.
Trace table	A technique for tracking the values of variables as an algorithm is executed step by step.
Input	Data that goes into an algorithm.
Processing	Operations performed on data (calculations, comparisons, decisions).
Output	Data that comes out of an algorithm.
Pseudo-code	A way of representing an algorithm using structured English and programming-style constructs, not specific to any programming language.
Flowchart	A visual representation of an algorithm using standard symbols connected by arrows.

AQA pseudo-code quick reference

Operation	AQA pseudo-code	Python
Assignment	x ← 5	x = 5
User input	x ← USERINPUT	x = input()
Output	OUTPUT x	print(x)
Integer division	x DIV y	x // y
Remainder (modulo)	x MOD y	x % y
For loop	FOR i ← 1 TO 5 ... ENDFOR	for i in range(1, 6):
While loop	WHILE condition ... ENDWHILE	while condition:
If statement	IF condition THEN ... ENDIF	if condition:
If-else	IF ... THEN ... ELSE ... ENDIF	if ... else:
String length	LEN(str)	len(str)
Array access	arr[0]	arr[0]
Logical AND	AND	and
Logical OR	OR	or
Logical NOT	NOT	not

Flowchart symbols

Symbol	Name	Purpose
	Terminator	Start or End of algorithm
	Process	Calculation, assignment, action
	Decision	Yes/No question (two exits)
	Input/Output	Data entering or leaving
	Flow line	Direction of flow

Trace table checklist

- Create one column for each variable in the algorithm
- Add an OUTPUT column if the algorithm produces output
- Work through the algorithm line by line
- Record the new value each time a variable changes
- For loops: repeat the loop body for each iteration
- For conditions: evaluate the condition and follow the correct branch
- Check your final values match the expected output

Typical pitfalls

Pitfall 1 — Incomplete algorithm definition.
Wrong: "An algorithm is a set of instructions."
Right: A sequence of steps that can be followed to complete a task. Include "sequence", "steps" and "task".

Pitfall 2 — Confusing algorithm and program.
Wrong: "An algorithm is a computer program."
Right: An algorithm is a design/plan. A program is an implementation of an algorithm in code.

Pitfall 3 — Incomplete decomposition definition.
Wrong: "Breaking a problem into parts."
Right: Breaking into sub-problems where each accomplishes an identifiable task, which might be further subdivided.

Pitfall 4 — Wrong FOR loop range in Python.

Wrong: FOR i ← 1 TO 5 becomes for i in range(1, 5)
Right: FOR i ← 1 TO 5 becomes for i in range(1, 6) — Python range excludes the end value.

Pitfall 5 — Forgetting DIV and MOD translations.
Wrong: Using / for DIV in Python.
Right: DIV = // (integer division), MOD = % (remainder).

Pitfall 6 — Skipping trace table rows.
Wrong: Jumping to the final answer without showing each step.
Right: Show every change to every variable. Marks are often awarded for intermediate values.

Pitfall 7 — Wrong order of operations in loops.
Wrong: Updating variables in the wrong order.
Right: Follow the exact sequence of statements. If x uses y, then y changes, the order matters.

Pitfall 8 — Misreading Boolean conditions.
Wrong: Assuming > means >= or vice versa.
Right: 12 > 12 is False. 12 >= 12 is True. Read operators carefully.

Pitfall 9 — Forgetting to OUTPUT at the end.
Wrong: Stopping the trace before the OUTPUT statement.
Right: Always include the final output in your trace table or answer.

Pitfall 10 — Not following exam instructions.
Wrong: Writing Python when asked for pseudo-code.
Right: Read the question carefully. If it asks for pseudo-code, use pseudo-code. If it asks for Python, use Python.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

WPE is free for everyone. If our resources help you or your students, you can support the project here.
© 2026 Williams Physics Education. All rights reserved.